

NOTES DE COURS

VERSION CONDENSÉE 6 HEURES

# Analyse de Données avec Python pour Professionnels de la Santé

*Six modules d'une heure – du Python de base au machine learning clinique*

|                     |                                                                              |
|---------------------|------------------------------------------------------------------------------|
| <b>Public ciblé</b> | Professionnels de santé, épidémiologistes, étudiants en santé publique       |
| <b>Prérequis</b>    | Aucun en programmation. Quelques notions de statistique de base sont utiles. |
| <b>Format</b>       | Notes accompagnant 6 notebooks Jupyter                                       |
| <b>Langue</b>       | Français                                                                     |
| <b>Outils</b>       | Python 3.10+, Jupyter / Google Colab                                         |

# Table des matières

|                                                           |           |
|-----------------------------------------------------------|-----------|
| <b>Avant-propos</b>                                       | <b>3</b>  |
| <b>1 Module 1 – Python et Pandas pour la santé</b>        | <b>5</b>  |
| 1.1 Pourquoi Python pour les données de santé             | 5         |
| 1.2 Variables et les quatre types fondamentaux            | 5         |
| 1.3 Collections : listes et dictionnaires                 | 5         |
| 1.4 Conditions et boucles : appliquer une règle clinique  | 6         |
| 1.5 Fonctions : emballer une formule clinique             | 6         |
| 1.6 Pandas : passer du Python pur à la table              | 6         |
| 1.7 Filtrer, trier, grouper                               | 6         |
| 1.8 Créer une colonne calculée                            | 7         |
| 1.9 Points clés du Module 1                               | 7         |
| <b>2 Module 2 – Nettoyage et statistique descriptive</b>  | <b>8</b>  |
| 2.1 Le nettoyage des données est un raisonnement clinique | 8         |
| 2.2 Les trois mécanismes de données manquantes            | 8         |
| 2.3 Détecter et compter les valeurs manquantes            | 8         |
| 2.4 Stratégies d'imputation                               | 8         |
| 2.5 Validation par plages cliniquement plausibles         | 9         |
| 2.6 Tendance centrale et dispersion                       | 9         |
| 2.7 Mesures épidémiologiques de base                      | 9         |
| 2.8 Pipeline de nettoyage réutilisable                    | 10        |
| 2.9 Points clés du Module 2                               | 10        |
| <b>3 Module 3 – Visualisation pour données de santé</b>   | <b>11</b> |
| 3.1 Une figure répond à une question                      | 11        |
| 3.2 Règles non-négociables                                | 11        |
| 3.3 Histogrammes et estimation de densité                 | 11        |
| 3.4 Boîtes à moustaches                                   | 12        |
| 3.5 Lignes : tendances temporelles et courbes épidémiques | 12        |
| 3.6 Nuages de points et associations                      | 12        |
| 3.7 Le tableau de bord multi-panneaux                     | 12        |
| 3.8 Points clés du Module 3                               | 12        |
| <b>4 Module 4 – Tests d'hypothèses</b>                    | <b>13</b> |
| 4.1 La logique d'un test                                  | 13        |
| 4.2 Ce qu'une p-value n'est pas                           | 13        |
| 4.3 Test t : un, deux, ou pairs                           | 13        |
| 4.4 Test du chi-deux d'indépendance                       | 14        |
| 4.5 L'alternative non paramétrique : Mann-Whitney U       | 14        |
| 4.6 Correction pour tests multiples                       | 14        |
| 4.7 Taille d'effet                                        | 14        |
| 4.8 Points clés du Module 4                               | 15        |
| <b>5 Module 5 – Régression</b>                            | <b>16</b> |
| 5.1 De la corrélation à la prédiction                     | 16        |
| 5.2 Régression linéaire simple et multiple                | 16        |
| 5.3 Facteur de confusion : brute vs ajustée               | 16        |
| 5.4 Vérifier les hypothèses                               | 16        |

|          |                                                                           |           |
|----------|---------------------------------------------------------------------------|-----------|
| 5.5      | Régression logistique pour les issues binaires . . . . .                  | 17        |
| 5.6      | Évaluation : matrice de confusion et les quatre taux . . . . .            | 17        |
| 5.7      | Points clés du Module 5 . . . . .                                         | 17        |
| <b>6</b> | <b>Module 6 – Machine learning pour la prédiction clinique</b>            | <b>18</b> |
| 6.1      | Quand ne pas utiliser le ML . . . . .                                     | 18        |
| 6.2      | Train / test stratifié . . . . .                                          | 18        |
| 6.3      | Validation croisée $k$ -folds . . . . .                                   | 18        |
| 6.4      | Arbres et forêts aléatoires . . . . .                                     | 18        |
| 6.5      | Courbe ROC et AUC . . . . .                                               | 19        |
| 6.6      | Calibration : les probabilités sont-elles dignes de confiance ? . . . . . | 19        |
| 6.7      | Audit d'équité . . . . .                                                  | 19        |
| 6.8      | Critères pour déployer un modèle . . . . .                                | 19        |
| 6.9      | Points clés du Module 6 . . . . .                                         | 20        |
| <b>7</b> | <b>Pour aller plus loin</b>                                               | <b>21</b> |
|          | <b>Annexe – Aide-mémoire</b>                                              | <b>22</b> |
|          | <b>Bibliographie</b>                                                      | <b>23</b> |

## Avant-propos

### Ce que ces notes sont – et ne sont pas

Ces notes accompagnent les six notebooks Jupyter de la **version condensée 6 heures** du cours *Analyse de données de santé avec Python*. Elles développent la matière théorique et méthodologique. Les notebooks contiennent le code complet et exécutable ; ces notes contiennent les **concepts**, les **formules**, les **contextes d'usage** et les **pièges** – avec seulement quelques courts extraits de code pour illustrer.

Si vous lisez ces pages *avant* la séance, vous saurez de quoi on va parler et pourquoi. Si vous les relisez *après*, elles servent d'aide-mémoire compact.

### Pourquoi 6 heures

Un cours détaillé de 30 heures existe (dossier `code/notebooks/` du dépôt). Mais pour une journée de formation, un atelier de trois demi-journées, ou un cours d'introduction, six heures sont souvent ce qu'on peut donner – et ce qu'un apprenant peut absorber en un seul bloc.

La version 6 h se concentre sur **l'épine dorsale analytique** :

1. **Python et Pandas** – le socle pour les cinq modules suivants.
2. **Nettoyage et statistique descriptive** – la base de toute analyse crédible.
3. **Visualisation** – communiquer un résultat.
4. **Tests d'hypothèses** – l'inférence formelle.
5. **Régression** – l'ajustement multivariable.
6. **Machine learning clinique** – prédiction, ROC, calibration, équité.

Trois sujets de la version 30 h ne sont **pas** couverts ici : analyse temporelle approfondie, analyse spatiale (géopandas), et projet capstone. Une brève mention de ces sujets en fin de document indique où les trouver.

### Comment utiliser ce document avec les notebooks

Le ratio recommandé est **15-20 minutes de lecture pour 40 minutes de manipulation du notebook**. Pour chaque module :

1. Lire la section correspondante de ces notes (environ 6-8 pages).
2. Ouvrir le notebook `module_ON_*.ipynb` dans Jupyter ou Colab.
3. Exécuter chaque cellule en lisant les commentaires.
4. Faire les exercices, puis comparer avec les cellules *Solution*.

#### NOTE

Les notebooks sont repérés par un en-tête graphique qui indique le numéro du module, la durée, et le niveau. La page d'accueil `index.ipynb` liste les 6 modules avec un bouton « Ouvrir dans Colab » pour chacun.

## Conventions typographiques

- **code** – terme technique, nom de variable, fonction Python.
- *italique* – mise en relief d'un concept la première fois qu'il est introduit.
- Les encarts colorés signalent : **Astuce** (vert), **Attention** (orange), **Note** (bleu), **Contexte clinique** (vert plus dense).

## 1. Module 1 – Python et Pandas pour la santé

### 1.1 Pourquoi Python pour les données de santé

Trois raisons font que Python s'est imposé comme l'outil par défaut dans la recherche en santé contemporaine :

**Reproductibilité.** Un script documente chaque étape de votre analyse. Quand un examinateur demande « comment avez-vous traité les valeurs de laboratoire manquantes ? », vous montrez le code – pas une description de clics dans des menus. Les revues expérimentales s'orientent toutes dans cette direction.

**Échelle.** Excel commence à peiner sur 100 000 lignes. Pandas en traite des millions sans difficulté. Plus important encore : « l'analyse est séparée des données », vous pouvez exécuter le même script sur un échantillon de 1 000 patients puis sur la base entière sans rien changer.

**Écosystème.** Des bibliothèques telles que `lifelines` (analyse de survie), `statsmodels` (modèles statistiques), `scikit-learn` (machine learning), `geopandas` (cartographie), `pingouin` (statistiques cliniques en français) offrent des outils que SPSS ou Stata n'égale pas. Et le tout est gratuit.

### 1.2 Variables et les quatre types fondamentaux

Une *variable* est un nom qui pointe vers une valeur. Vous n'avez **pas à déclarer** le type : Python le devine à partir de la valeur affectée.

```
age = 45                                # int    -- nombre entier
temperature = 37.2                      # float  -- nombre à virgule
diagnostic = "Diabete de type 2"        # str    -- texte entre guillemets
hospitalise = True                       # bool   -- True ou False
```

Les quatre types couvrent l'essentiel des données de santé :

| Type  | Stocke           | Exemple clinique                   |
|-------|------------------|------------------------------------|
| int   | Nombre entier    | Âge, nombre d'admissions, parité   |
| float | Nombre à virgule | Température, HbA1c, créatinine     |
| str   | Texte            | Code CIM-10, identifiant patient   |
| bool  | Vrai / Faux      | Fumeur ? Hospitalisé ? Grossesse ? |

### 1.3 Collections : listes et dictionnaires

Une *liste* range plusieurs valeurs **dans l'ordre**. Une série de tensions chez un même patient, par exemple, est naturellement une liste.

```
tensions = [120, 135, 128, 142, 118]    # mmHg, syst.
```

L'indexation commence à **zéro** : `tensions[0]` est la première valeur, `tensions[-1]` la dernière. La syntaxe `tensions[1:3]` extrait les indices 1 et 2 (la borne droite est exclue, c'est une convention mathématique clé en Python).

Un *dictionnaire* associe des **clés** à des **valeurs** – exactement la forme d'une fiche patient.

```
patient = {
    "id": "PAT-0042",
    "age": 58,
    "sexe": "F",
```

```

    "tension_systolique": 148,
    "medicaments": ["Amlodipine", "HCTZ"],
}
patient["age"]    # -> 58

```

**NOTE**

Une **cohorte** = une **liste de dictionnaires**. C'est exactement la forme que pandas convertit en `DataFrame` : la fonction `pd.DataFrame(cohorte)` construit le tableau directement.

## 1.4 Conditions et boucles : appliquer une règle clinique

Une *conditionnelle* encode une règle de décision. Une *boucle* l'applique à chaque patient d'une cohorte.

```

for p in cohorte:
    if p["hba1c"] >= 6.5:
        statut = "Diabetique"
    elif p["hba1c"] >= 5.7:
        statut = "Pre-diabetique"
    else:
        statut = "Normal"

```

L'**indentation** – quatre espaces – est la façon de Python de délimiter les blocs. Pas d'accolades, contrairement à C, Java ou R. Une indentation incorrecte lève une `IndentationError`.

L'ordre des conditions importe. On teste d'abord la plus stricte ( $\geq 6,5$ ), puis la suivante. L'inversé classerait tout patient avec HbA1c  $\geq 5,7$  comme « pré-diabétique » – y compris les diabétiques avérés.

## 1.5 Fonctions : emballer une formule clinique

Une *fonction* est un bloc de code nommé, qui prend des **arguments** et renvoie une **valeur**. Une fois écrite, on l'utilise comme une formule.

```

def imc(poids_kg: float, taille_m: float) -> float:
    """Indice de Masse Corporelle en kg/m^2."""
    return poids_kg / (taille_m ** 2)

```

Trois bonnes pratiques :

1. **Docstring**. La chaîne entre triple guillemets juste après `def` documente la fonction. `help(imc)` l'affiche.
2. **Annotations de type** (`poids_kg: float`). Elles documentent l'intention sans rien forcer à l'exécution. Vivement recommandées en contexte clinique.
3. **Appels par mot-clé**. Préférez `imc(poids_kg=82, taille_m=1.75)` à `imc(82, 1.75)`.

**ATTENTION**

Pour toute fonction de calcul médical (dosage, eGFR, scores), les appels positionnels sont une source d'erreur grave. Une inversion entre `poids_kg` et `age` dans une formule de dosage mg/kg peut être dramatique. **Toujours** préférer les appels par mot-clé.

## 1.6 Pandas : passer du Python pur à la table

*Pandas* est la bibliothèque standard pour manipuler des tableaux en Python. L'objet central est le `DataFrame` : pensez à un tableur, mais programmable.

```
import pandas as pd
gm = pd.read_csv("gapminder.tsv", sep="\t")
```

Avant toute analyse, exécuter les **cinq méthodes d'inspection** fait gagner des heures :

| Méthode                             | Ce qu'elle vous dit                   |
|-------------------------------------|---------------------------------------|
| <code>df.head(n)</code>             | Les $n$ premières lignes              |
| <code>df.shape</code>               | (nb lignes, nb colonnes)              |
| <code>df.dtypes</code>              | Type de chaque colonne                |
| <code>df.describe()</code>          | Moyenne, $\sigma$ , quartiles         |
| <code>df['x'].value_counts()</code> | Fréquences d'une colonne catégorielle |

## 1.7 Filtrer, trier, grouper

Le **filtrage** utilise un *masque booléen* – une colonne de `True/False` qu'on place entre crochets.

```
afrique = gm[gm["continent"] == "Africa"]
```

Pour combiner deux conditions, utilisez `&` (et), `|` (ou), `~` (non), et **paranthésisez chaque condition**. Cette dernière obligation est non-négociable : sans parenthèses, Python applique les opérateurs dans le mauvais ordre.

```
pauvre_africain = gm[
    (gm["continent"] == "Africa")
    & (gm["gdpPercap"] < 1000)
]
```

Le **tri** se fait avec `.sort_values("col")`. Pour obtenir les  $n$  plus grandes/petites valeurs directement, `.nlargest(n, "col")` / `.nsmallest(n, "col")`.

Le **groupby** est le *geste pivot* de pandas. Le patron est « **split, apply, combine** » (McKinney, 2017) :

1. **Split** – couper les lignes en groupes selon une catégorie.
2. **Apply** – calculer une statistique sur chaque groupe.
3. **Combine** – recoller le tout en une table.

```
gm.groupby("continent")["lifeExp"].mean()
```

## 1.8 Créer une colonne calculée

Pandas est *vectorisé* : on ne fait pas de boucle pour remplir une colonne. L'opération est appliquée à toutes les lignes en même temps.

```
gm["gdp_milliards"] = gm["pop"] * gm["gdpPercap"] / 1e9
```

Pour transformer une variable continue en catégories ordonnées, `pd.cut` :

```
gm["categorie_ev"] = pd.cut(
    gm["lifeExp"],
    bins=[0, 50, 65, 75, 100],
    labels=["Tres faible", "Faible", "Moyen", "Eleve"],
)
```

)

## 1.9 Points clés du Module 1

- Quatre types Python suffisent pour 95 % des données cliniques.
- Une cohorte = liste de dictionnaires = `DataFrame`.
- Pandas se manipule par *méthodes enchaînées* : `df.filter(...).groupby(...).agg(...)`.
- Les masques booléens demandent toujours des **parenthèses** autour de chaque condition.
- `pd.cut` est l'outil universel pour catégoriser une variable continue (seuils cliniques, tranches d'âge).

## 2. Module 2 – Nettoyage et statistique descriptive

### 2.1 Le nettoyage des données est un raisonnement clinique

Aucune base de données réelle n'est propre. Dans les dossiers patient informatisés (DPI), une étude de référence (Weiskopf & Weng, 2013) estime qu'environ 18 % des valeurs de laboratoire présentent un problème de qualité : doublon, incohérence d'unité, valeur impossible.

Nettoyer, c'est **prendre des décisions**. Supprimer une ligne ? Imputer une valeur ? Marquer un *flag* ? Chaque choix déplace l'analyse. La discipline consiste à **documenter chaque décision** pour qu'un relecteur puisse les reproduire.

### 2.2 Les trois mécanismes de données manquantes

Le cadre conceptuel a été établi par Rubin (1976) et n'a pas changé depuis. Trois mécanismes :

**MCAR – Missing Completely At Random.** Le manquement est indépendant de tout. Un tube de prélèvement qui casse, par exemple. Supprimer les lignes concernées est **sans biais** ; on perd seulement de la puissance.

**MAR – Missing At Random.** Le manquement dépend de variables **observées**. Exemple : le cholestérol n'est pas mesuré chez les patients jeunes (l'âge est observé). L'imputation **conditionnelle à l'âge** est sans biais.

**MNAR – Missing Not At Random.** Le manquement dépend de la valeur **non observée** elle-même. Les patients les plus malades ratent leur consultation de suivi, donc on n'a pas leur tension à six mois. **C'est le cas le plus dangereux** : supprimer biaise systématiquement vers les patients en meilleure santé.

#### ATTENTION

Dans les DPI, MNAR est la **règle**, pas l'exception. Avant d'imputer ou de supprimer, vérifiez **toujours** si l'issue (la variable dont vous voulez parler) diffère entre les lignes complètes et les lignes à données manquantes. Si oui, le manquement est informatif : il faut le modéliser, pas le balayer.

### 2.3 Détecter et compter les valeurs manquantes

Pandas utilise NaN (Not a Number) pour le numérique manquant et None pour les objets. Les deux sont détectés par `.isnull()`.

```
df.isnull().sum()           # nb manquant par colonne
df.isnull().any(axis=1)     # True/False par ligne
```

Un point souvent négligé : vérifier les manquements **simultanés**. La corrélation de la matrice `df.isnull()` révèle si deux colonnes manquent **ensemble**, indice possible d'une cause commune (la même consultation ratée, le même automate en panne).

### 2.4 Stratégies d'imputation

Règle pratique : **médiane par défaut** pour tout ce qui est clinique. La majorité des variables de laboratoire ont des queues droites (créatinine, protéine C-réactive, ferritine). La moyenne y est tirée par les extrêmes.

Pour les variables critiques (issue principale, exposition clé), gardez un **indicateur de manquement** *avant* d'imputer :

| Méthode                 | Quand l'utiliser                               |
|-------------------------|------------------------------------------------|
| Médiane                 | Continu asymétrique (lab, tension, IMC)        |
| Moyenne                 | Continu symétrique (taille adulte)             |
| Mode                    | Catégoriel (sexe, statut tabagique)            |
| Par sous-groupe         | Valeur attendue dépend d'un groupe connu       |
| KNN                     | Plusieurs colonnes corrélées manquent ensemble |
| MICE / IterativeImputer | Imputation multivariée, plusieurs runs         |

```
df["tension_manquait"] = df["tension"].isnull().astype(int)
df["tension"] = df["tension"].fillna(df["tension"].median())
```

L'indicateur peut ensuite servir de prédicteur dans un modèle (le fait que la valeur ait été absente peut être pronostique en soi).

## 2.5 Validation par plages cliniquement plausibles

Toute base de données a des valeurs **absurdes** : un âge de  $-3$  ans, une tension de 450 mmHg, une HbA1c de 55 (lue comme NGSP alors qu'elle est en IFCC). Codez vos plages de plausibilité en tant que dictionnaire, appliquez-les systématiquement :

```
PLAGES = {
    "age": (0, 120),
    "tension": (60, 300),
    "hba1c": (2.0, 20.0),
}
```

Les valeurs hors plage sont **presque toujours** des erreurs de saisie – soit une faute de frappe (450 au lieu de 145), soit une confusion d'unité (5,5 mmol/L de glycémie lu comme 5,5 mg/dL).

### CONTEXTE CLINIQUE

Le cas des unités mérite une mention spéciale. La glycémie en mg/dL et mmol/L sont à un facteur 18 près :  $100 \text{ mg/dL} \approx 5,5 \text{ mmol/L}$ . Sans colonne d'unité explicite, vous ne pouvez pas distinguer « 7 » pour une glycémie à jeun « normale en mmol/L » de « pathologique en mg/dL ». La vérité absolue : **exigez une colonne unité dès la collecte.**

## 2.6 Tendances centrale et dispersion

**Moyenne ou médiane ?** Règle pragmatique : si une valeur extrême déplace la moyenne de plus de quelques pourcents, rapportez la **médiane**. La médiane est robuste : une valeur aberrante à 340 mg/dL au milieu de glycémies de 90-110 ne la déplace presque pas.

**Pour la dispersion**, l'écart-type ( $\sigma$ ) est l'outil par défaut pour les distributions symétriques. Pour les distributions *asymétriques* (durée de séjour, coûts hospitaliers, biologie à longue queue), l'**intervalle interquartile (IQR = Q3 – Q1)** est plus honnête.

Le **coefficient de variation**  $CV = \sigma/\mu$  (en %) est sans unité. Utile pour comparer la dispersion relative de variables aux unités différentes : « l'hémoglobine varie moins ( $CV \approx 5\%$ ) que la glycémie ( $CV \approx 8\%$ ) chez un même patient ».

## 2.7 Mesures épidémiologiques de base

Quatre mesures couvrent l'essentiel des rapports en santé publique :

**Prévalence** – proportion de cas **existants** à un instant  $t$  :

$$P = \frac{\text{nombre de cas}}{\text{population totale}}.$$

Rapporter toujours avec un **intervalle de confiance**. Pour une proportion proche de 0 ou 1, préférer le *Wilson* ou *Agresti-Coull* à l'approximation normale (Wald).

**Taux d'incidence** – nouveaux cas par personne-temps :

$$IR = \frac{\text{nouveaux cas}}{\text{personnes-années suivies}}.$$

Pour des événements rares, le compte suit une loi de Poisson, d'où  $SE(IR) = \sqrt{cas}/PA$ .

**Ratio de risque (RR)** et **ratio de cotes (OR)** à partir d'un tableau  $2 \times 2$  :

|            | Malade+ | Malade– |
|------------|---------|---------|
| Exposé     | $a$     | $b$     |
| Non exposé | $c$     | $d$     |

$$RR = \frac{a/(a+b)}{c/(c+d)}, \quad OR = \frac{a \cdot d}{b \cdot c}.$$

L'OR est l'estimé dans les études **cas-témoin** ; le RR dans les études de **cohorte**. Pour de petits effets et issues rares,  $OR \approx RR$ . Dès que la prévalence dépasse  $\sim 10\%$ , l'OR *surestime* le RR – rappez-le comme OR, pas comme RR.

Les intervalles de confiance pour RR et OR sont calculés sur l'échelle ln (la distribution y est approximativement normale), puis exponentialisés.

## 2.8 Pipeline de nettoyage réutilisable

Le bon réflexe : bundler les étapes dans **une seule fonction**, sur le modèle entrée  $\rightarrow$  sortie. Testée une fois, elle s'applique à tous les jeux de données similaires.

```
def nettoyer(df, plages=PLAGES):
    df = df.copy()
    df.columns = df.columns.str.strip().str.lower()
    df = df.drop_duplicates()
    for col, (lo, hi) in plages.items():
        if col in df.columns:
            df.loc[(df[col] < lo) | (df[col] > hi), col] = np.nan
    for col in df.select_dtypes(include="number").columns:
        df[col] = df[col].fillna(df[col].median())
    return df
```

## 2.9 Points clés du Module 2

- Les trois mécanismes MCAR / MAR / MNAR déterminent ce qu'on peut faire sans biais.
- En DPI, MNAR est la règle : vérifier que les lignes complètes et incomplètes ont la même issue.
- Médiane par défaut pour le clinique ; IQR pour la dispersion asymétrique.
- Prévalence, incidence, RR, OR – toujours rapporter avec IC.
- Encapsuler le nettoyage dans une fonction.

### 3. Module 3 – Visualisation pour données de santé

#### 3.1 Une figure répond à une question

Avant d'écrire la moindre ligne de code de visualisation, formulez la **question** à laquelle la figure répond. « Distribution de l'IMC » n'est pas une question. « La distribution de l'IMC diffère-t-elle entre les groupes d'âge ? » en est une.

Le type de question détermine le type de graphique.

| Question                                            | Type de graphique  |
|-----------------------------------------------------|--------------------|
| Comment une variable est-elle distribuée ?          | Histogramme, KDE   |
| Comment une variable diffère-t-elle entre groupes ? | Boite à moustaches |
| Comment compare-t-on des comptages ?                | Barres             |
| Comment évolue-t-elle dans le temps ?               | Ligne              |
| Deux variables continues sont-elles associées ?     | Nuage de points    |
| Quelles variables sont corrélées ?                  | Heatmap            |

#### 3.2 Règles non-négociables

Toute figure publiée ou présentée en réunion respecte ces règles de base. Les violer trahit un manque de rigueur que vos lecteurs remarqueront :

1. **Les axes portent leur unité.** « Tension systolique (mmHg) », jamais juste « BP ».
2. **Les barres commencent à zéro.** Tronquer l'axe vertical pour amplifier visuellement une différence est l'un des mensonges visuels les plus documentés ([Tufté, 2001](#)).
3. **Pas de 3D.** Aucun graphique 3D n'a jamais amélioré la lisibilité. Ils introduisent des distorsions de perception.
4. **Pas de camemberts au-delà de 3 parts.** L'œil humain compare mal les angles ; les barres font mieux.
5. **Pas de double axe Y.** Sauf cas exceptionnel, ils suggèrent fausse corrélation et permettent de manipuler la perception.
6. **Palette accessible aux daltoniens.** Suivez les recommandations de [Wong \(2011\)](#).
7. **Annoter** la taille d'échantillon ( $n = 1\,234$ ) et les **seuils cliniques** (lignes horizontales / verticales aux cut-offs pertinents).

#### 3.3 Histogrammes et estimation de densité

L'histogramme montre comment une variable continue est répartie. Le nombre de classes (**bins**) compte : trop peu cache la structure, trop transforme le graphe en bruit. Une règle empirique :  $k \approx \sqrt{n}$ .

```
ax.hist(donnees, bins=20, edgecolor="white")
ax.axvline(126, color="red", ls="--",
          label="Seuil diabete (126 mg/dL)")
```

Pour comparer plusieurs distributions sur les mêmes axes, l'estimation *kernel density* (KDE) est plus lisible : courbes lissées au lieu de barres se chevauchant.

### 3.4 Boîtes à moustaches

Anatomie : la boîte va de  $Q_1$  à  $Q_3$ , la barre médiane est le  $Q_2$ , les moustaches s'étendent au dernier point dans  $1,5 \times IQR$ , les points au-delà sont les valeurs aberrantes individuelles.

Limite importante : **les boîtes cachent la bimodalité**. Deux distributions très différentes peuvent produire la même boîte. En cas de doute, superposer un *stripplot* ou montrer l'histogramme en parallèle.

### 3.5 Lignes : tendances temporelles et courbes épidémiques

Le patron canonique pour une épidémie : **barres journalières + moyenne mobile sur 7 jours** en haut, **courbe cumulée** en bas. La barre montre le bruit réel, la ligne le signal lissé.

La moyenne mobile sur 7 jours efface la périodicité hebdomadaire (déclarations repérables en bloc le lundi, vacances scolaires, etc.).

```
epi["mm_7j"] = epi["nouveaux_cas"].rolling(7).mean()
```

### 3.6 Nuages de points et associations

Le scatter plot affiche deux variables continues sur un plan. Pour encoder une troisième variable, deux choix s'offrent : la **taille des points** (bubble chart) ou la **couleur**. La taille marche bien pour des grandeurs sur plusieurs ordres – la population des pays, par exemple.

L'échelle **logarithmique** sur un axe est cruciale dès que la variable s'étale sur 3-4 ordres de grandeur (PIB par habitant, charges virales, comptages bactériens). Sans le log, 95 % des points s'agglutinent dans un coin.

### 3.7 Le tableau de bord multi-panneaux

Un tableau de bord est une **seule figure** avec plusieurs panneaux qui répondent ensemble à une question générale (« que sait-on de la santé mondiale en 2007 ? »).

Règles complémentaires pour les multi-panneaux :

1. **Lettrer** les panneaux (A, B, C, D) pour les référencer dans la légende et le texte.
2. **Garder le même code couleur** entre panneaux : un continent gardé en bleu dans le panneau A doit rester bleu en B.
3. **Hiérarchie de l'œil** : panneau A en haut à gauche reçoit l'œil en premier ; mettez-y l'image la plus importante.

### 3.8 Points clés du Module 3

- Le type de graphique dépend du type de question, pas de l'esthétique.
- Palette colour-blind-safe par défaut (Wong 2011).
- Annoter unités,  $n$ , seuils cliniques.
- Boîte à moustaches : utile pour comparer, dangereuse pour l'unimodalité.
- Tableau de bord = une question, plusieurs panneaux complémentaires.

## 4. Module 4 – Tests d’hypothèses

### 4.1 La logique d’un test

Un test d’hypothèse est une *preuve par contradiction probabiliste* :

1. On suppose la **nulle**  $H_0$  **vraie** (pas d’effet).
2. On calcule une statistique de test à partir des données.
3. On se demande : *si  $H_0$  était vraie, ces données seraient-elles inhabituelles ?* Cette probabilité est la **p-value**.
4. Si la p-value est  $< \alpha$  (souvent 0,05), on **rejette**  $H_0$ .

### 4.2 Ce qu’une p-value n’est pas

Aucune confusion n’a coûté plus cher à la recherche en santé que les interprétations erronées de la p-value. La déclaration officielle de l’American Statistical Association ([Wasserstein & Lazar, 2016](#)) en énumère les plus dommageables. Répétons les essentielles :

- Ce n’est **pas** la probabilité que  $H_0$  soit vraie.
- Ce n’est **pas** la probabilité que le résultat soit dû au hasard.
- Elle ne mesure **pas** la taille d’un effet.
- Un test significatif sur un grand  $n$  peut correspondre à un effet **cliniquement négligeable**.

#### ATTENTION

La règle d’or : **toujours rapporter une p-value avec une taille d’effet et un intervalle de confiance**. Sur 100 000 patients, un essai peut détecter une différence moyenne de 0,5 mmHg avec  $p < 10^{-6}$ . Statistiquement significatif, cliniquement non pertinent.

### 4.3 Test t : un, deux, ou pairs

**Test t à un échantillon.** La moyenne d’un échantillon diffère-t-elle d’une valeur de référence connue ? Exemple : le cholestérol moyen dans notre clinique diffère-t-il de la moyenne nationale de 200 mg/dL ?

**Test t à deux échantillons indépendants.** Deux groupes indépendants diffèrent-ils ? Exemple : la tension dans le groupe traité diffère-t-elle de celle du placebo ?

Conditions : indépendance, normalité dans chaque groupe (assoupli par le TCL pour  $n \gtrsim 30$ ), **égalité des variances**. Cette dernière se relâche avec la **correction de Welch** (`equal_var=False`), qui est le **défaut moderne recommandé**. Pratique : toujours mettre `equal_var=False` ([Ruxton, 2006](#)).

**Test t apparié.** Les mêmes patients mesurés avant et après une intervention. Le test utilise les **différences intra-patient**, éliminant la variabilité inter-patient et augmentant la puissance.

```
from scipy import stats
t, p = stats.ttest_ind(traitement, controle, equal_var=False)
```

## 4.4 Test du chi-deux d'indépendance

Pour deux variables **catégorielles**, le chi-deux compare les comptages observés aux comptages attendus sous indépendance :

$$\chi^2 = \sum_{ij} \frac{(O_{ij} - E_{ij})^2}{E_{ij}}.$$

Condition : chaque effectif attendu doit être  $\geq 5$ . Sinon utiliser le **test exact de Fisher** (`stats.fisher_exact`).

## 4.5 L'alternative non paramétrique : Mann-Whitney U

Quand la variable est asymétrique (durée de séjour, coûts) ou que  $n$  est petit, le test t sur l'échelle brute peut induire en erreur. Mann-Whitney U compare les **rangs** : il teste si une distribution domine stochastiquement l'autre.

### NOTE

Comparer Mann-Whitney U et un test t après transformation log : souvent les deux donnent des conclusions similaires sur les données asymétriques. La transformation log a l'avantage de rendre les coefficients de régression interprétables (en pourcentage de changement).

## 4.6 Correction pour tests multiples

Si vous testez **20 hypothèses indépendantes au seuil 0,05 sous  $H_0$** , vous attendez en moyenne **un faux positif par hasard**. Sur 1 000 tests sans effet réel, 50 sont « significatifs » à  $p < 0,05$ . C'est l'*erreur familiale*.

Deux familles de corrections :

**Bonferroni** (contrôle le *family-wise error rate*) : multiplier chaque  $p$  par  $m$  (ou comparer à  $\alpha/m$ ). Très conservateur ; approprié quand même **un** faux positif est inacceptable (essais cliniques, décisions réglementaires).

**Benjamini-Hochberg FDR** (Benjamini & Hochberg, 1995) : contrôle la proportion attendue de faux positifs parmi les positifs déclarés. Standard en génomique et en analyses exploratoires.

```
from statsmodels.stats.multitest import multipletests
_, p_corr, _, _ = multipletests(pvals, method="fdr_bh")
```

## 4.7 Taille d'effet

Le  $d$  de Cohen :

$$d = \frac{\bar{x}_1 - \bar{x}_2}{s_{pooled}}.$$

Repères indicatifs (Cohen, 1988) :

| $ d $       | Interprétation |
|-------------|----------------|
| $< 0,2$     | Négligeable    |
| $0,2 - 0,5$ | Petit          |
| $0,5 - 0,8$ | Moyen          |
| $\geq 0,8$  | Large          |

Ces repères ont été établis dans le domaine des sciences sociales et doivent être interprétés avec

prudence en clinique : une différence « petite » sur la mortalité cardiovasculaire peut être très importante au niveau populationnel.

#### 4.8 Points clés du Module 4

- p-value  $\neq$  probabilité que  $H_0$  soit vraie.
- Toujours rapporter taille d'effet *et* IC à côté de la p-value.
- Welch (`equal_var=False`) par défaut pour le test t à deux échantillons.
- Mann-Whitney U pour les distributions asymétriques.
- Tests multiples : Bonferroni pour ne tolérer aucun FP ; BH-FDR pour les screenings.

## 5. Module 5 – Régression

### 5.1 De la corrélation à la prédiction

La régression est l'outil le plus utilisé en recherche clinique contemporaine. Elle répond à **deux questions distinctes** qu'il faut savoir séparer.

**Inférentielle** : quels facteurs sont associés à une issue, en ajustant pour les autres ? Exemple : l'IMC est-il associé à la tension, indépendamment de l'âge et du sexe ? L'outil de prédilection est `statsmodels`, qui donne coefficients, IC 95 %, p-values,  $R^2$ .

**Prédictive** : pour un nouveau patient, quelle est la probabilité d'issue ? Quel est son risque à 10 ans ? L'outil de prédilection est `scikit-learn`, qui donne prédictions et métriques de performance.

#### NOTE

`statsmodels` et `scikit-learn` ne sont **pas redondants**. Le même modèle (régression linéaire ou logistique) ne vous renverra pas la même chose : `statsmodels` pour interpréter, `scikit-learn` pour prédire. Apprenez à utiliser les deux.

### 5.2 Régression linéaire simple et multiple

Le modèle linéaire :

$$y_i = \beta_0 + \beta_1 x_{1i} + \beta_2 x_{2i} + \dots + \varepsilon_i, \quad \varepsilon_i \sim \mathcal{N}(0, \sigma^2).$$

Chaque coefficient  $\beta_k$  s'interprète comme le **changement attendu de  $y$  pour une augmentation de une unité de  $x_k$ , en maintenant les autres prédicteurs constants**. C'est la clé de l'ajustement pour facteur de confusion.

### 5.3 Facteur de confusion : brute vs ajustée

Un facteur de confusion est associé à l'**exposition** et à l'**issue**, sans être sur le chemin causal entre les deux. Si on l'ignore, l'estimation brute est biaisée.

Exemple-école : le café semble être un facteur de risque cardiovasculaire dans une analyse brute. Mais une fois qu'on ajuste pour le tabagisme, le coefficient du café s'évanouit. Le tabagisme était le *vrai* facteur, et il était associé à la consommation de café – d'où l'illusion.

La régression multiple ajuste *simultanément* pour tous les confondants observés. Elle ne corrige **pas** les confondants inobservés : pour cela il faut des méthodes causales avancées (variables instrumentales, score de propension, *difference-in-differences*).

### 5.4 Vérifier les hypothèses

Quatre conditions pour des inférences valides :

1. **Linéarité** – la relation entre  $x_k$  et  $y$  est linéaire. Diagnostic : graphique résidus vs prédits.
2. **Indépendance** – les observations sont indépendantes (faux si données longitudinales : utiliser un modèle mixte).
3. **Homoscédasticité** – la variance des résidus est constante. Un nuage en entonnoir est le drapeau rouge.

4. **Normalité des résidus** – diagnostic QQ-plot. Pas besoin d'être parfait pour  $n$  grand (TCL).

### 5.5 Régression logistique pour les issues binaires

Quand l'issue est oui/non, l'OLS prédit des valeurs en dehors de  $[0, 1]$  – non utilisable. La logistique modélise la **probabilité** :

$$\Pr(y = 1 | x) = \sigma(\beta_0 + \beta_1 x_1 + \dots), \quad \sigma(z) = \frac{1}{1 + e^{-z}}.$$

Les coefficients sont des **log-odds**. Pour les transformer en **odds ratios** interprétables :  $OR = e^\beta$ . Un OR de 1,04 par mg/dL de glycémie signifie « chaque mg/dL multiplie les cotes de diabète par 1,04 ». Sur une plage de 30 mg/dL, l'effet composé est  $1,04^{30} \approx 3$  – substantiel.

### 5.6 Évaluation : matrice de confusion et les quatre taux

Sur un échantillon test, comparer prédictions et vérité dans une matrice  $2 \times 2$  :

|        | Prédit + | Prédit – |
|--------|----------|----------|
| Réel + | VP       | FN       |
| Réel – | FP       | VN       |

Les quatre métriques clés :

- **Sensibilité** =  $VP/(VP + FN)$ . Capacité à **détecter** les malades. Cruciale pour les outils de *rule-out* (« si test négatif, je sais que ce n'est pas ça »).
- **Spécificité** =  $VN/(VN + FP)$ . Capacité à **exclure** les non-malades. Cruciale pour les outils de *rule-in*.
- **VPP** (valeur prédictive positive) =  $VP/(VP + FP)$ . Quand le test est positif, quelle est la probabilité d'être vraiment malade.
- **VPN** =  $VN/(VN + FN)$ . Quand le test est négatif, quelle est la probabilité d'être vraiment sain.

#### CONTEXTE CLINIQUE

Sensibilité et spécificité dépendent uniquement du **test**. VPP et VPN dépendent aussi de la **prévalence dans la population testée**. Le même test, appliqué en population générale (faible prévalence) puis aux urgences (forte prévalence), donne des VPP / VPN très différents. C'est pourquoi les recommandations de dépistage dépendent du contexte épidémiologique local.

### 5.7 Points clés du Module 5

- **statsmodels** pour l'inférence ; **scikit-learn** pour la prédiction.
- Coefficient = effet net « toutes choses égales par ailleurs », parmi les variables observées.
- $OR = e^\beta$  pour la logistique.
- Toujours ajuster pour les confondants **observés**.
- Sensibilité / spécificité dépendent du test ; VPP / VPN dépendent de la prévalence.

## 6. Module 6 – Machine learning pour la prédiction clinique

### 6.1 Quand ne pas utiliser le ML

Le machine learning est séduisant. Il est aussi **surutilisé**. Trois situations où il est inapproprié :

**Petits jeux de données** ( $n < 200$ ). Les arbres et les forêts surajustent. La régression logistique avec quelques prédicteurs choisis cliniquement est plus fiable et généralise mieux.

**Validation réglementaire**. La FDA, l'EMA, l'AFSSAPS et la plupart des agences exigent des modèles **interprétables**. Un modèle boîte-noire ne passera pas l'évaluation. La régression logistique avec quelques prédicteurs bien justifiés est le standard de fait.

**Une règle simple marche déjà**. « Glycémie  $> 126$  mg/dL » identifie l'essentiel des diabétiques. « INR  $> 4$  » identifie le surdosage en AVK. Si une règle a une bonne sensibilité, elle est plus simple, plus auditable et plus enseignable qu'un random forest.

#### NOTE

Une revue systématique récente ([Christodoulou et al., 2019](#)) a montré que sur 71 études comparant ML à la régression logistique en prédiction clinique, **aucun bénéfice consistant en performance n'a été démontré** pour le ML. La régression logistique reste un baseline très difficile à battre.

### 6.2 Train / test stratifié

Toujours **stratifier** sur l'issue. Sans stratification, le jeu test peut avoir une prévalence très différente du jeu d'entraînement et les métriques deviennent instables.

```
from sklearn.model_selection import train_test_split
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.3,
    random_state=42, stratify=y
)
```

### 6.3 Validation croisée $k$ -folds

Pour les petits jeux, le test unique est bruyant. La **validation croisée** divise les données en  $k$  plis, entraîne sur  $k - 1$ , teste sur le restant, et moyenne les  $k$  scores obtenus.

Cinq plis est le défaut classique. Dix plis donne des estimations moins biaisées mais plus bruyantes. Pour les outcomes rares (incidence  $< 5\%$ ), utiliser une **validation croisée stratifiée** (StratifiedKFold) qui préserve la prévalence dans chaque pli.

### 6.4 Arbres et forêts aléatoires

Un *arbre de décision* est une séquence de questions oui/non aboutissant à une feuille avec une classe prédite. Sa force : la **lisibilité clinique**. Un clinicien peut suivre l'arbre et expliquer la prédiction au lit du patient.

Sa faiblesse : sans contrainte de profondeur, l'arbre mémorise les données d'entraînement et généralise mal. **Toujours fixer max\_depth**.

Une *forêt aléatoire* combine des centaines d'arbres entraînés sur des échantillons *bootstrap* avec des sous-ensembles aléatoires de variables, puis moyenne leurs prédictions. La randomisation décorrèle les arbres et réduit la variance.

Trade-off : meilleures prédictions, mais on perd la lisibilité d'« un arbre » pointable.

## 6.5 Courbe ROC et AUC

La courbe ROC trace la **sensibilité** contre  $1 - \text{spécificité}$  quand on fait varier le seuil de décision de 0 à 1. L'**AUC** (Area Under Curve) résume tout en un nombre.

| AUC       | Interprétation                         |
|-----------|----------------------------------------|
| 0,5       | Pas mieux qu'un tirage                 |
| 0,7 – 0,8 | Acceptable                             |
| 0,8 – 0,9 | Excellent                              |
| > 0,9     | Exceptionnel (rare en clinique réelle) |

### ATTENTION

L'AUC seule ne suffit pas. Un modèle peut avoir une excellente AUC **et être inutile** si ses probabilités sont mal calibrées, ou s'il présente un biais entre sous-groupes. **Toujours** compléter l'AUC par une courbe de calibration et un audit d'équité.

## 6.6 Calibration : les probabilités sont-elles dignes de confiance ?

Une bonne AUC n'implique **pas** de bonnes probabilités. Un modèle **bien calibré** produit des probabilités qui correspondent aux fréquences observées : si vous prédisiez 30 % de risque à 10 ans, environ 30 % des patients développent effectivement la maladie.

On vérifie avec :

- **Courbe de calibration** – probabilité moyenne prédite vs proportion observée, par décile de risque. Une bonne calibration s'approche de la diagonale.
- **Score de Brier** – erreur quadratique moyenne entre probabilités prédites et issues 0/1. Plus bas = mieux.

## 6.7 Audit d'équité

Vérifier que le modèle performe **de manière comparable** entre groupes (sexe, âge, ethnie, comorbidité). Un écart de plusieurs points d'AUC entre groupes est un signal d'alerte.

[Obermeyer et al. \(2019\)](#) ont démontré qu'un algorithme largement déployé aux États-Unis sous-allouait systématiquement les soins aux patients afro-américains – non par dessein, mais parce que la dépense passée servait de proxy au besoin futur, et les patients afro-américains dépensaient moins à niveau de morbidité égal. L'audit d'équité est un **contrôle minimal** avant tout déploiement.

## 6.8 Critères pour déployer un modèle

Avant de mettre un modèle en production clinique :

1. **Validation externe** sur un jeu indépendant (autre hôpital, autre période). La performance baisse souvent de plusieurs points d'AUC – attendez-vous-y.
2. **Audit d'équité** déjà discuté.
3. **Calibration** explicite, avec recalibration si nécessaire.

4. **Définition du seuil de décision** co-construite avec les cliniciens et liée à une action concrète.
5. **Monitoring** en production pour détecter le drift.
6. **Documentation TRIPOD-AI** ([Collins et al., 2024](#)).

## 6.9 Points clés du Module 6

- Le ML n'est pas toujours le bon outil ; la logistique reste un baseline difficile à battre.
- Stratifier le train/test ; valider en CV  $k$ -folds.
- AUC est nécessaire mais pas suffisant : ajouter calibration et audit d'équité.
- Validation externe avant tout déploiement clinique.
- Documenter selon TRIPOD-AI.

## 7. Pour aller plus loin

Cette version 6 heures se concentre sur l'épine dorsale analytique. La version 30 heures couvre trois sujets non abordés ici :

**Analyse temporelle approfondie.** Décomposition saisonnière des séries de cas, modélisation ARIMA, lissage exponentiel, courbes épidémiques avec retards de déclaration. Voir `code/notebooks/ch09_geosp`

**Analyse spatiale.** GeoPandas, projections, choropleths, auto-corrélation spatiale, hotspot analysis avec les indices de Moran et Getis-Ord. Voir le même chapitre.

**Projet capstone.** Analyse complète end-to-end sur un jeu réel, rédaction d'un rapport et soutenance orale. Voir `code/notebooks/ch10_projet_final.ipynb`.

Ressources additionnelles incontournables :

- **An Introduction to Statistical Learning** (James, Witten, Hastie, Tibshirani) – gratuit en ligne : <https://www.statlearning.com/>.
- **Python Data Science Handbook** (Jake VanderPlas) – gratuit en ligne : <https://jakevdp.github.io/PythonDataScienceHandbook/>.
- **Regression Modeling Strategies** (Frank Harrell) – la bible de la régression en biostatistique.
- **TRIPOD-AI Statement** pour le rapportage de modèles de prédiction en santé : <https://www.tripod-statement.org/>.

## Annexe – Aide-mémoire des snippets clés

### Charger un fichier

```
df = pd.read_csv("fichier.csv")
df = pd.read_csv("fichier.tsv", sep="\t")
df = pd.read_excel("fichier.xlsx", sheet_name=0)
```

### Inspecter

```
df.head(); df.shape; df.dtypes; df.describe()
df["col"].value_counts(); df["col"].nunique()
df.isnull().sum()
```

### Filtrer / trier / grouper

```
df[df["col"] > 5]
df[(df["a"] > 1) & (df["b"] == "X")]
df.sort_values("col", ascending=False)
df.nlargest(10, "col")
df.groupby("cat")["x"].agg(["mean", "median"])
```

### Créer une colonne

```
df["nouvelle"] = df["a"] * df["b"]
df["categ"] = pd.cut(df["x"], bins=[0, 1, 2], labels=["L", "H"])
```

### Nettoyer

```
df = df.drop_duplicates()
df = df.dropna(subset=["col"])
df["col"] = df["col"].fillna(df["col"].median())
df.loc[df["col"] < 0, "col"] = np.nan
```

### Visualiser (matplotlib + seaborn)

```
fig, ax = plt.subplots(figsize=(8, 5))
ax.hist(s, bins=30); ax.set_xlabel("..."); ax.set_ylabel("...")
sns.boxplot(data=df, x="cat", y="val", ax=ax)
ax.axvline(seuil, color="red", ls="--")
plt.tight_layout(); plt.show()
```

### Tests

```
from scipy import stats
stats.ttest_ind(a, b, equal_var=False)      # Welch
stats.ttest_rel(avant, apres)              # apparie
stats.chi2_contingency(table)              # chi2
stats.mannwhitneyu(g1, g2, alternative="two-sided")
```

### Régression

```
import statsmodels.api as sm
X = sm.add_constant(df[predicteurs])
ols = sm.OLS(df["y"], X).fit()
logit = sm.Logit(df["y_bin"], X).fit(dispatch=False)
np.exp(logit.params)                      # OR
```

## Machine learning

```
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import roc_auc_score, confusion_matrix

X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y)
rf = RandomForestClassifier(n_estimators=200, max_depth=6,
                           random_state=42).fit(X_tr, y_tr)
roc_auc_score(y_te, rf.predict_proba(X_te)[: , 1])
```

## Bibliographie

- Benjamini, Y. & Hochberg, Y. (1995). Controlling the false discovery rate : a practical and powerful approach to multiple testing. *Journal of the Royal Statistical Society B*, 57(1), 289-300.
- Christodoulou, E. et al. (2019). A systematic review shows no performance benefit of machine learning over logistic regression for clinical prediction models. *Journal of Clinical Epidemiology*, 110, 12-22.
- Cohen, J. (1988). *Statistical Power Analysis for the Behavioral Sciences*, 2e ed. Lawrence Erlbaum.
- Collins, G. S. et al. (2024). TRIPOD+AI statement : updated guidance for reporting clinical prediction models that use regression or machine learning methods. *BMJ*, 385, e078378.
- Obermeyer, Z., Powers, B., Vogeli, C., & Mullainathan, S. (2019). Dissecting racial bias in an algorithm used to manage the health of populations. *Science*, 366(6464), 447-453.
- Rubin, D. B. (1976). Inference and missing data. *Biometrika*, 63(3), 581-592.
- Ruxton, G. D. (2006). The unequal variance t-test is an underused alternative to Student's t-test and the Mann-Whitney U test. *Behavioral Ecology*, 17(4), 688-690.
- Tufte, E. R. (2001). *The Visual Display of Quantitative Information*, 2e ed. Graphics Press.
- Wasserstein, R. L. & Lazar, N. A. (2016). The ASA statement on p-values : context, process, and purpose. *The American Statistician*, 70(2), 129-133.
- Weiskopf, N. G. & Weng, C. (2013). Methods and dimensions of electronic health record data quality assessment. *Journal of the American Medical Informatics Association*, 20(1), 144-151.
- Wong, B. (2011). Color blindness. *Nature Methods*, 8(6), 441.